

Отчет по лабораторной 4 “Сегментация объектов”

Для выполнения лабораторной работы, была взята модель mobilenet_v2 в силу своей легковесности, а также датасет собак имеющий следующую структуру:

📁 / ... / OpenCVLab / DogSegmentation /	
Name	
📁 test	
📁 train	
📁 val	

Рисунок 1 — Структура датасета

📁 / ... / DogSegmentation / train /	
Name	
📁 images	
📁 masks	

Рисунок 2- Разделение на картинки и маски



Рисунок 3 — Пример изображения



Рисунок 4 — Пример маски

```
] import os
import cv2
import torch
import numpy as np
from torch.utils.data import Dataset, DataLoader
import segmentation_models_pytorch as smp
DEBUG=False
```

Рисунок 5 — Импорт библиотек

```

class SegmentationDataset(Dataset):
    def __init__(self, images_dir, masks_dir, transform=None):
        self.images_dir = images_dir
        self.masks_dir = masks_dir
        self.images = os.listdir(images_dir)
        self.transform = transform

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        img_name = self.images[idx]
        img_path = os.path.join(self.images_dir, img_name)
        # Ensure mask filename matches or handle extension replacement
        mask_name = os.path.splitext(img_name)[0] + ".jpg"
        mask_path = os.path.join(self.masks_dir, mask_name)
        #debug
        if mask_path=="DogSegmentation/train/masks/.ipynb_checkpoints.jpg":
            mask_path="DogSegmentation/train/masks/dog.8838.jpg"
            img_path="DogSegmentation/train/images/dog.8838.jpg"
        if DEBUG:
            print(mask_path+"@"+img_path)
        #endif
        image = cv2.imread(img_path)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        # Read mask as grayscale
        mask = cv2.imread(mask_path, cv2.IMREAD_GRAYSCALE)

        # Simple resizing and tensor normalization
        image = cv2.resize(image, (256, 256))
        mask = cv2.resize(mask, (256, 256), interpolation=cv2.INTER_NEAREST)

        # Convert to PyTorch Tensors
        image = torch.tensor(image, dtype=torch.float32).permute(2, 0, 1) / 255.0
        mask = torch.tensor(mask, dtype=torch.long) # Shape: [H, W]

        # Add channel dimension to mask if doing binary segmentation
        mask = mask.unsqueeze(0).float()

        return image, mask

```

Рисунок 6 — Код датасета

```

# 2. Setup Data Loaders
train_dataset = SegmentationDataset("DogSegmentation/train/images", "DogSegmentation/train/masks")
train_loader = DataLoader(train_dataset, batch_size=8, shuffle=True)

# 3. Create Lightweight Model (MobileNetV2 backbone)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = smp.Unet(
    encoder_name="mobilenet_v2",      # Lightweight mobile encoder
    encoder_weights="imagenet",      # Transfer Learning weights
    in_channels=3,                   # RGB input images
    classes=1                        # 1 for binary segmentation mask
).to(device)

# 4. Optimizer and Loss Function
optimizer = torch.optim.Adam(model.parameters(), lr=(1e-4*4))
# Combination Loss works best for Lightweight segmentation
criterion = smp.losses.DiceLoss(mode='binary')

```

Рисунок 7 — Импорт модели и датасета

```

[18]: # 5. Training Loop
model.train()
epochs = 100

print(f"Starting training on device: {device}")
for epoch in range(epochs):
    epoch_loss = 0
    for images, masks in train_loader:
        images, masks = images.to(device), masks.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, masks)
        loss.backward()
        optimizer.step()

        epoch_loss += loss.item()

    print(f"Epoch {epoch+1}/{epochs} - Loss: {epoch_loss/len(train_loader):.4f}")

# 6. Save Weights directly to PyTorch File
torch.save(model.state_dict(), "lightweight_unet_mobilenet.pth")
print("Training complete! Model saved.")

Starting training on device: cpu
Epoch 1/100 - Loss: 0.9944
Epoch 2/100 - Loss: 0.9929
Epoch 3/100 - Loss: 0.9923
Epoch 4/100 - Loss: 0.9921
Epoch 5/100 - Loss: 0.9922
Epoch 6/100 - Loss: 0.9922
Epoch 7/100 - Loss: 0.9922
Epoch 8/100 - Loss: 0.9921
Epoch 9/100 - Loss: 0.9921

```

Рисунок 8- Обучение модели

```
Epoch 93/100 - Loss: 0.9919
Epoch 94/100 - Loss: 0.9918
Epoch 95/100 - Loss: 0.9918
Epoch 96/100 - Loss: 0.9918
Epoch 97/100 - Loss: 0.9918
Epoch 98/100 - Loss: 0.9918
Epoch 99/100 - Loss: 0.9918
Epoch 100/100 - Loss: 0.9918
Training complete! Model saved.
```

```
[2]: # Тестовая выборка (shuffle=False, чтобы порядок изображений не менялся)
test_dataset = SegmentationDataset('DogSegmentation/test/images', 'DogSegmentation/test/masks')
test_loader = DataLoader(test_dataset, batch_size=4, shuffle=False)
```

```
[3]: print("\nНачало тестирования модели...")
# 1. Создаем точно такую же конфигурацию модели, которая обучалась
model = smp.Unet(
    encoder_name="mobilenet_v2",          # имя энкодера (должно совпадать с оригиналом)
    encoder_weights=None,                 # ставим None, так как веса загрузим из файла
    in_channels=3,                        # количество входных каналов
    classes=1,                           # количество классов для сегментации
)

# 2. Определяем устройство (GPU, если доступно, иначе CPU)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# 3. Загружаем словарь весов из файла
# map_location принудительно отправляет веса на нужное устройство (важно, если веса обучались на GPU, а запускаются на CPU)
weights_path = "lightweight_unet_mobilenet.pth"
state_dict = torch.load(weights_path, map_location=device)
# 5. Загружаем веса в саму модель
model.load_state_dict(state_dict)
model.eval() # Переводим модель в режим оценки (выключает dropout и batchnorm)

test_loss = 0
total_dice_score = 0
# Отключаем расчет градиентов для экономии памяти и ускорения
with torch.no_grad():
```

Рисунок 9- Перевод модели в режим тестирования

```
test_loss = 0
total_dice_score = 0
# Отключаем расчет градиентов для экономии памяти и ускорения
with torch.no_grad():
    for images, masks in test_loader:
        images, masks = images.to(device), masks.to(device)

        # Прямой проход (логиты модели)
        outputs = model(images)

        # Считаем функцию потерь на тесте
        loss = criterion(outputs, masks)
        test_loss += loss.item()

        # Переводим логиты в вероятность (сигмоида) и бинаризируем (по порогу 0.5)
        preds = torch.sigmoid(outputs)
        preds = (preds > 0.5).float()

        # Считаем метрику Dice Score вручную для батча
        intersection = (preds * masks).sum(dim=(2, 3))
        union = preds.sum(dim=(2, 3)) + masks.sum(dim=(2, 3))
        # Добавляем 1e-7 во избежание деления на ноль
        dice = (2. * intersection + 1e-7) / (union + 1e-7)
        total_dice_score += dice.mean().item()

avg_test_loss = test_loss / len(test_loader)
avg_dice_score = total_dice_score / len(test_loader)

print(f"Результаты на тестовой выборке:")
print(f"-> Средняя потеря (Dice Loss): {avg_test_loss:.4f}")
print(f"-> Средняя точность (Dice Score / F1): {avg_dice_score:.4f}")
```

Начало тестирования модели...

Рисунок 10- Код тестирования модели

Результаты на тестовой выборке:
-> Средняя потеря (Dice Loss): 0.9920
-> Средняя точность (Dice Score / F1): 1.9777

```
[4]: import matplotlib.pyplot as plt
```

```
[5]: # =====  
# 6. ВИЗУАЛИЗАЦИЯ ПРЕДСКАЗАНИЙ ИЗ ТЕСТА  
# =====  
# Возьмем один батч из тестовой выборки для отрисовки графиков  
images, masks = next(iter(test_loader))  
model.eval()  
with torch.no_grad():  
    preds = torch.sigmoid(model(images.to(device))).cpu().numpy()  
  
# Отрисуем первые 3 изображения из тестового батча  
num_samples = min(10, len(images))  
fig, axes = plt.subplots(num_samples, 3, figsize=(12, 4 * num_samples))  
  
for i in range(num_samples):  
    # Восстанавливаем оси из тензора (C, H, W) в (H, W, C) для matplotlib  
    img_show = images[i].permute(1, 2, 0).numpy()  
    gt_mask = masks[i][0].numpy()  
    pred_mask = (preds[i][0] > 0.5).astype(np.float32)  
  
    # Если картинка одна, axes будет одномерным массивом  
    ax_row = axes[i] if num_samples > 1 else axes  
  
    ax_row[0].imshow(img_show)  
    ax_row[0].set_title("Оригинал (Тест)")  
    ax_row[0].axis("off")  
  
    ax_row[1].imshow(gt_mask, cmap="gray")  
    ax_row[1].set_title("Истинная маска")  
    ax_row[1].axis("off")  
  
    ax_row[2].imshow(pred_mask, cmap="gray")  
    ax_row[2].set_title("Предсказание модели")  
    ax_row[2].axis("off")  
  
plt.tight_layout()  
plt.show()
```

Рисунок 11- Код визуализации результата



Рисунок 12- Результат тестирования



Рисунок 13- Результат тестирования